

Análisis Físico y de Protocolo de la Comunicación Serial RS232 mediante MicroPython en la Raspberry Pi Pico 2W

Torres Freddy 1401641, Pérez Andrés Francois1401638
{est.freddym.torres, est.andresf.perez}@unimilitar.edu.co
Docente: Jose Rugeles

Resumen — En esta práctica de laboratorio se analizó de manera experimental y teórica el comportamiento del protocolo de comunicación serial asíncrona UART/RS232 en la capa física. Utilizando un microcontrolador Raspberry Pi Pico 2W programado en entorno MicroPython, se generaron tramas de datos modificando parámetros fundamentales de la comunicación como la tasa de baudios (desde 1200 hasta 115200 bps), el número de bits de datos, y los esquemas de paridad (par, impar y sin paridad). Mediante el uso de un osciloscopio digital, se midieron los tiempos de bit individuales y la duración total de las tramas de caracteres específicos (como "E", "x", "@", "p", "\$"), contrastando los resultados experimentales con las predicciones del modelo matemático del protocolo. Adicionalmente, se caracterizó el comportamiento temporal al transmitir bloques continuos de caracteres a bajas velocidades (600 baudios) y altas velocidades (57600 baudios). Los resultados permitieron evidenciar la relación inversa entre la tasa de baudios y el tiempo de bit, así como la variación en la eficiencia del canal al alterar la estructura de la trama de datos.

Abstract — In this laboratory practice, the physical layer behavior of the UART/RS232 asynchronous serial communication protocol was analyzed both experimentally and theoretically. Using a Raspberry Pi Pico 2W microcontroller programmed in a MicroPython environment, data frames were generated by modifying fundamental communication parameters such as the baud rate (from 1200 to 115200 bps), the number of data bits, and parity schemes (even, odd, and no parity). By utilizing a digital oscilloscope, individual bit times and the total duration of frames for specific characters (such as "E", "x", "@", "p", "\$") were measured, contrasting the experimental results with the protocol's mathematical model. Additionally, the temporal behavior was characterized when transmitting continuous blocks of characters at low speeds (600 baud) and high speeds (57600 baud). The results demonstrated the inverse relationship between the baud rate and the bit time, as well as the variation in channel efficiency when altering the structure of the data frame.

Palabras clave — Comunicación asíncrona, RS232, UART, Tiempo de bit, Tasa de baudios, Bit de paridad, MicroPython, Raspberry Pi Pico 2W.

Keywords — Asynchronous communication, RS232, UART, Bit time, Baud rate, Parity bit, MicroPython, Raspberry Pi Pico 2W.

I. INTRODUCCIÓN

En el desarrollo de sistemas de telecomunicaciones modernos, la comunicación serial asíncrona mediante el protocolo UART (Universal Asynchronous Receiver-Transmitter) sigue siendo uno de los pilares fundamentales para el intercambio de información a corta distancia entre microcontroladores, sensores y computadoras periféricas. A pesar del surgimiento de protocolos síncronos de alta velocidad, la UART destaca por su simplicidad de implementación, requiriendo únicamente dos líneas físicas de datos (transmisión TX y recepción RX) y una referencia común de tierra (GND). Sin embargo, para garantizar la integridad de los datos en un canal de comunicación real, no basta con comprender el protocolo a nivel de software; es imperativo analizar el comportamiento de las señales en la capa física (el dominio del tiempo y el voltaje). Parámetros como la tasa de baudios (velocidad de modulación) definen de manera estricta la duración temporal de cada bit transmitido, lo que impone exigencias técnicas tanto a los circuitos integrados como a los instrumentos de medición encargados de capturar estas señales transitorias.

El presente informe detalla el estudio experimental de la estructura de la trama de datos del estándar RS232 y su emulación mediante UART en una Raspberry Pi Pico 2W (basada en el microcontrolador RP2350). A través de la programación en MicroPython, se generaron escenarios de prueba variando de forma sistemática la velocidad, la longitud del carácter y el método de detección de errores por paridad. Para lograr un análisis integral, se plantearon los siguientes objetivos específicos:

1. Identificar y medir en el dominio del tiempo el ancho de un bit individual para diferentes tasas de baudios estándar utilizando cursores en el osciloscopio.

2. Analizar y dibujar la forma de onda eléctrica de caracteres ASCII específicos, identificando físicamente los bits de inicio, datos (LSB a MSB), paridad y parada.
3. Evaluar el impacto temporal y la variación de la eficiencia del canal al modificar la configuración de paridad (par, impar y sin paridad) y la cantidad de bits de parada.
4. Validar de forma analítica los tiempos teóricos de duración de tramas compuestas por bloques continuos de texto y contrastarlos con las mediciones experimentales obtenidas en el laboratorio.

II. MARCO TEÓRICO

A. Comunicación Serial Asíncrona y el Estándar RS232

A diferencia de la transmisión paralela, la comunicación serial transmite la información bit a bit, de forma secuencial, sobre un único conductor físico. Al ser asíncrona, no existe una línea de reloj común (Clock) compartida entre el transmisor y el receptor para sincronizar la lectura de los bits. En su lugar, ambos dispositivos deben acordar previamente una velocidad de transmisión constante (tasa de baudios) y confiar en una estructura de trama específica para sincronizar el inicio y fin de cada carácter.

El estándar RS232 define las especificaciones eléctricas (niveles de tensión) y mecánicas para este tipo de comunicación, donde tradicionalmente un 0 lógico (Space) se representa con voltajes positivos (+3V a +15V) y un 1 lógico (Mark) con voltajes negativos (-3V a -15V).

En los microcontroladores modernos, este protocolo se emula utilizando niveles lógicos TTL/CMOS, donde el estado de reposo (Idle) e intelectual 1 lógico corresponde a 3.3 (voltaje de alimentación) y el 0 lógico corresponde a 0V (GND).

B. Estructura de la Trama de Datos UART

Para asegurar que el receptor pueda identificar cuándo comienza y termina un carácter, cada palabra de datos se encapsula dentro de una "trama" que consta de los siguientes elementos en orden de transmisión temporal:

1. **Bit de Inicio (Start Bit):** Es un bit de transición que siempre se transmite en nivel lógico bajo (0 lógico o 0V). Rompe el estado de reposo (Idle) de la línea que se mantiene en alto (3.3V), indicando al receptor que comience a contar el tiempo para muestrear los bits de datos entrantes.
2. **Bits de Datos (Data Bits):** Corresponde al código binario del carácter ASCII transmitido (normalmente 7 u 8 bits). Se transmiten siempre comenzando por el bit menos significativo (LSB, *Least Significant Bit*) y terminando con el más significativo (MSB, *Most Significant Bit*).

3. **Bit de Paridad (Parity Bit):** Es un mecanismo básico de detección de errores de un solo bit en la capa física. Puede configurarse como:

- **Sin Paridad (None):** No se incluye este bit en la trama.
- **Paridad Par (Even):** El bit se calcula de modo que el número total de unos (1s) en los bits de datos más el de paridad sea un número par.
- **Paridad Impar (Odd):** El bit se calcula para que el número total de unos en la trama de datos más el de paridad sea impar.

4. **Bit de Parada (Stop Bit):** Retorna la línea a nivel lógico alto (1 lógico o 3.3V) durante una duración equivalente a 1, 1.5 o 2 tiempos de bit, garantizando que la línea quede en reposo antes de la siguiente transmisión.

C. Relación entre Tasa de Baudios y Tiempo de Bit

La tasa de baudios (f_{baud}) representa el número de símbolos (o bits en este caso) transmitidos por segundo. El tiempo de bit (t_b), que es la duración física en el tiempo de un solo pulso de nivel lógico alto o bajo en el cable, se define matemáticamente por la relación inversa:

$$t_b = \frac{1}{f_{\text{baud}}}$$

De igual manera, el tiempo total teórico que requiere la transmisión de una trama completa (T_{trama}) para un carácter individual se calcula multiplicando el número total de bits de la trama (N_{bits}) por el tiempo de bit:

$$T_{\text{trama}} = N_{\text{bits}} \times t_b = \frac{N_{\text{bits}}}{f_{\text{baud}}}$$

Donde N_{bits} se determina sumando:

$$N_{\text{bits}} = 1 \text{ (Inicio)} + \text{Bits de Datos} \\ + \text{Bit de Paridad (0 o 1)} \\ + \text{Bits de Parada (1 o 2)}$$

D. El Módulo UART en MicroPython (Raspberry Pi Pico 2W)

La Raspberry Pi Pico 2W incorpora el chip RP2350, el cual cuenta con dos periféricos UART de hardware independientes (UART0 y UART1). El firmware de MicroPython expone este hardware a través de la clase `machine.UART`, permitiendo configurar dinámicamente la velocidad (baudrate), el tamaño de palabra (bits), el tipo de paridad (parity) y los bits de parada (stop).

Una característica fundamental del RP2350 es la **multiplexación de pines**, la cual permite asignar las señales TX y RX de los bloques UART internos a diferentes pines físicos (GPIO) de la placa. Por defecto, UART0 se mapea en GP0 (TX, pin físico 1) y GP1 (RX, pin físico 2), lo cual facilita

el enrutamiento y la interconexión con instrumentos externos sin colisionar con otros periféricos.

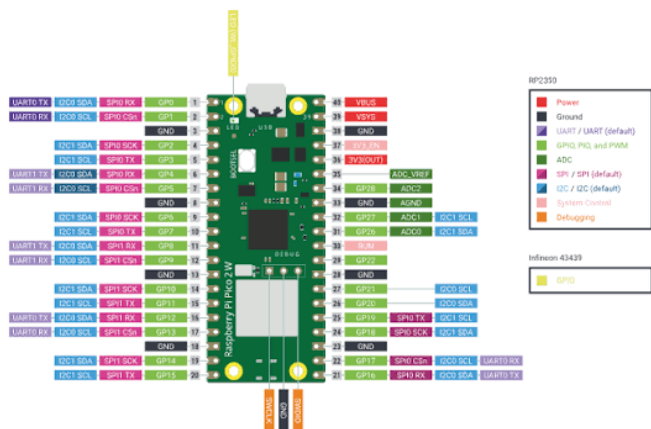


Fig. 1. Raspberry Pi Pico 2W

E. Consideraciones de Instrumentación para el Análisis de Protocolos

Para visualizar adecuadamente señales transitorias digitales de alta velocidad en el osciloscopio, se deben aplicar técnicas correctas de adquisición:

- **Configuración del Trigger (Disparo):** Debido a que el protocolo transmite de forma asíncrona a intervalos definidos por el software, se debe configurar un trigger de tipo *Flanco (Edge)* en modo de flanco de bajada (*Falling Edge*) asignado al canal de datos. Esto permite que el osciloscopio capture de manera precisa el momento exacto en que la línea cae de 3.3Va 0V (Bit de Inicio), estabilizando la imagen de la trama en la pantalla.
- **Uso de Cursores de Tiempo:** Permite medir de forma experimental el ancho temporal (Δt) de un pulso individual o de la trama completa para contrastarlo con el modelo teórico y determinar el error de reloj sistemático del hardware.

III. DESARROLLO DE LA PRACTICA

A. Configuración Inicial e Identificación de Terminales (Numerales 3.1 y 3.2)

Para el desarrollo de la práctica, se procedió inicialmente con la configuración del microcontrolador de la Raspberry Pi Pico 2W. Se descargó el firmware oficial correspondiente en formato .UF2 y, mediante la activación de la línea de arranque a través del botón físico *BOOTSEL*, se cargó el entorno de ejecución de MicroPython en la memoria del dispositivo. Posteriormente, se consultaron las especificaciones técnicas del hardware para identificar la asignación física de pines de la UART0. Se estableció que el Pin 1 (GP0) opera como la terminal de transmisión de datos (TX) y el Pin 3 (GND) como

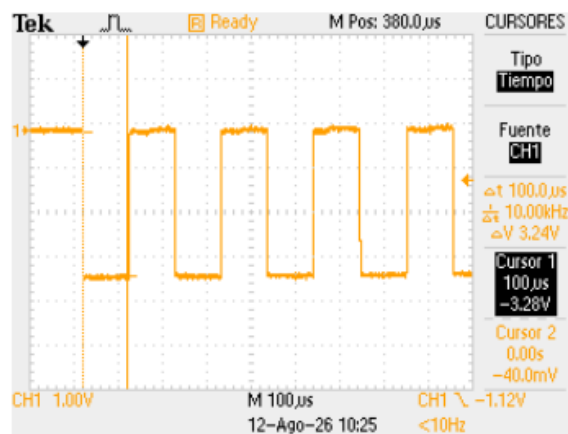
la referencia de tierra común. Se realizó la conexión física de una punta de prueba del Canal 1 de un osciloscopio digital Tektronix TDS 2012B directamente sobre el Pin 1 (TX), asegurando la pinza de tierra en el Pin 3 (GND) para capturar los transitorios de voltaje respecto a una referencia estable.

B. Envío de Caracteres ASCII y Calibración (Numeral 3.3)

Con el fin de validar el canal de comunicación física y calibrar el sistema de disparo del instrumento de medición, se diseñó un algoritmo básico en MicroPython. El código configura la UART0 con parámetros estándar: velocidad de transmisión de 9600 baudios, longitud de palabra de 8 bits de datos, sin paridad y 1 bit de parada (esquema 8-N-1). El programa transmite cíclicamente el carácter ASCII "U" (binario 01010101) con un intervalo de un segundo, conmutando de forma simultánea el estado del diodo LED integrado en la placa.

Debido a que la representación binaria de la letra "U" alterna de forma perfecta transiciones de nivel alto y bajo, la señal eléctrica capturada se comporta como una onda cuadrada periódica.

- **Medición del Tiempo de Bit:** Utilizando los cursores de tiempo del osciloscopio (ver Fig. 2), se midió la duración de un pulso individual en nivel bajo. El instrumento registró un intervalo experimental de 100.0 μ s. Al contrastarlo con el valor teórico para 9600 baudios (104.17 μ s), se obtuvo un error sistemático de apenas 4.00%, el cual corresponde a la limitación de resolución de la pantalla digital del osciloscopio en la base de tiempo de 100 μ s/div
- **Medición de Amplitud:** Se ajustó el osciloscopio para realizar mediciones con cursores de amplitud (ver Fig. 3). Se determinó una amplitud neta pico a pico de 3.36 V. Sabiendo que la placa opera con lógica LVTTTL nominal de 3.30 V, se registró un error marginal del 1.82%, confirmando la excelente regulación de voltaje del microcontrolador.



TDS 2012B - 9:48:33 a. m. 12/08/2026

Fig. 2. Medición temporal del tiempo de bit para el carácter "U" a 9600 baudios.

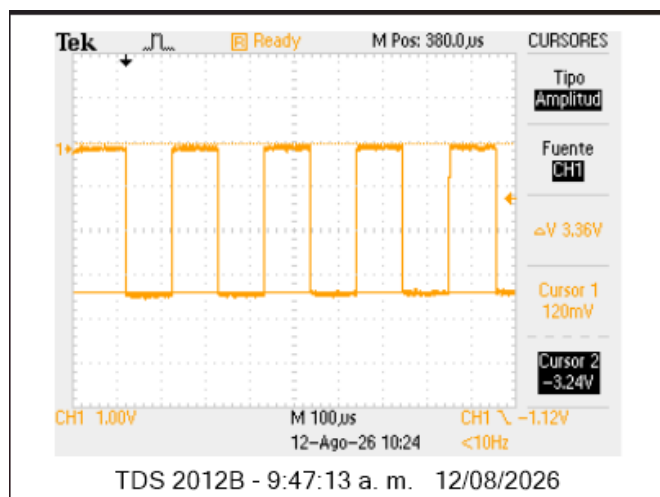


Fig. 3. Medición de la amplitud pico a pico de la señal.

C. Medida de los Tiempos de Bit y Variación de Baudios (Numeral 3.4)

Una vez calibrado el sistema, se evaluó la relación matemática inversa entre la tasa de transmisión (f_{baud}) y el tiempo físico de un bit (t_b). Para este experimento, se modificó el script para enviar el carácter ASCII "Y" (secuencia binaria 10011010). Para garantizar la validez de la medición, se ajustaron los cursores de tiempo estrictamente sobre el pulso más estrecho de la trama.

Se reprogramó dinámicamente el parámetro baudrate de la UART en seis frecuencias estandarizadas. Las mediciones de tiempo y niveles lógicos extraídas del osciloscopio se consolidan en la **Tabla I**.

TABLA I MEDICIÓN DE TIEMPOS DE BIT Y VOLTAJES EN UNA SEÑAL RS232

Baudios	Tiempo de bit (Teórico)	Tiempo de bit (Experimental)	% Error	Voltaje máximo	Voltaje mínimo
1200	833.33 μs	880.00 μs	5.60%	3.40 V	40.0 mV
4800	208.33 μs	220.00 μs	5.60%	3.40 V	40.0 mV
9600	104.17 μs	108.00 μs	3.67%	3.40 V	40.0 mV

38400	26.04 μs	27.00 μs	3.69%	3.24 V	-40.0 mV
57600	17.36 μs	18.40 μs	5.99%	3.28 V	0.00 V
115200	8.68 μs	9.60 μs	10.60%	3.24 V	0.00 V

A continuación, se presenta la progresión gráfica de las mediciones del tiempo de bit (Δt) para cada una de las velocidades evaluadas:

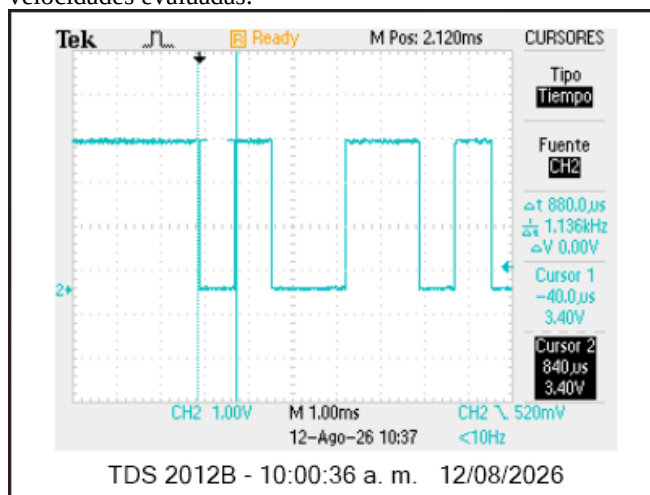


Fig. 4. Medición de tiempo de bit a 1200 baudios ($\Delta t = 880.0 \mu\text{s}$). Base de tiempo: 1.00 ms/div.

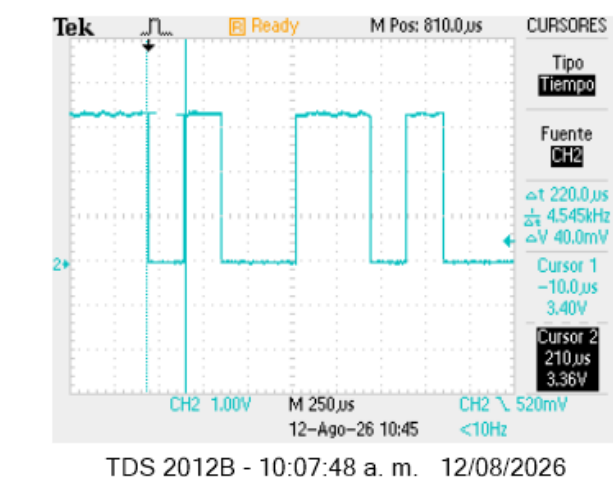


Fig. 5. Medición de tiempo de bit a 4800 baudios ($\Delta t = 220.0 \mu\text{s}$). Base de tiempo: 250 μs /div.

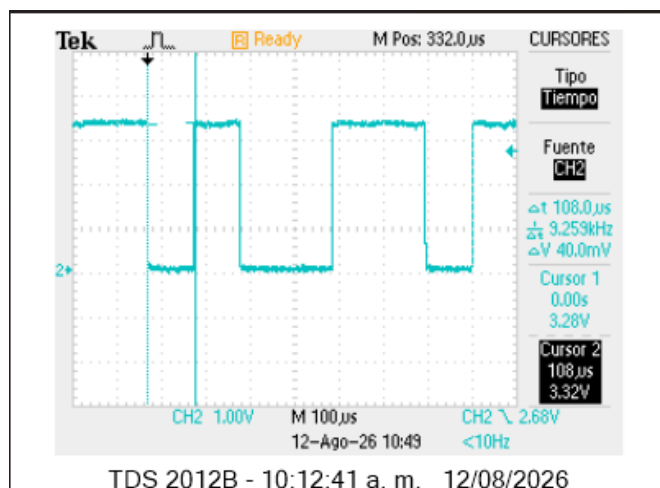


Fig. 6. Medición de tiempo de bit a 9600 baudios ($\Delta t = 108.0\mu s$). Base de tiempo: $100\mu s/div$.

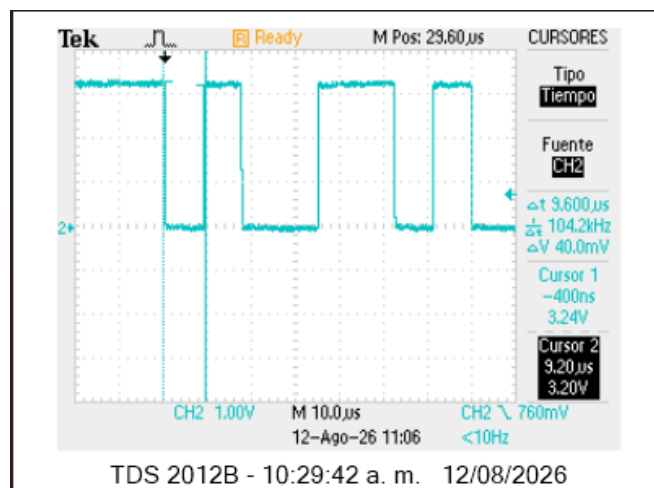


Fig. 9. Medición de tiempo de bit a 115200 baudios ($\Delta t = 9.60\mu s$). Base de tiempo: $10.0\mu s/div$.

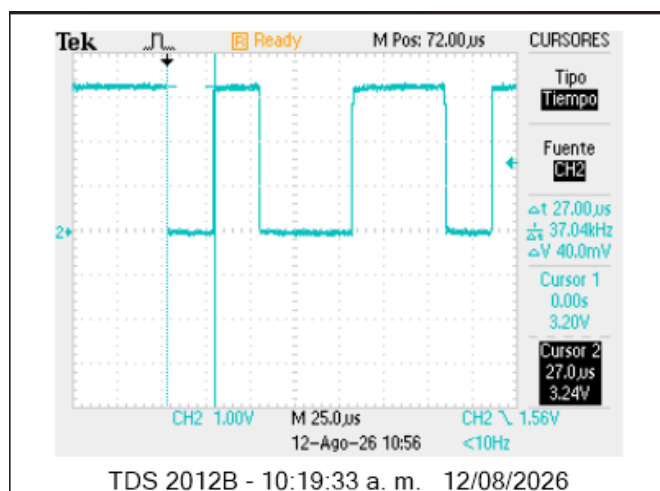


Fig. 7. Medición de tiempo de bit a 38400 baudios ($\Delta t = 27.00\mu s$). Base de tiempo: $25.0\mu s/div$

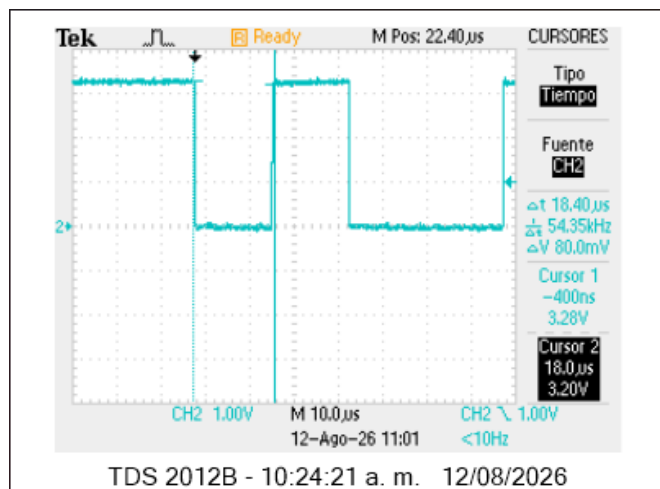


Fig. 8. Medición de tiempo de bit a 57600 baudios ($\Delta t = 18.40\mu s$). Base de tiempo: $10.0\mu s/div$.

1) Análisis de Resultados en el Dominio Temporal:

La formulación teórica establece que el tiempo de bit es $t_b = 1/f_{baud}$. Al contrastar esta ecuación con los datos empíricos de la Tabla I y la secuencia gráfica, se comprueba categóricamente que la relación de decaimiento temporal se cumple físicamente. La compresión de la señal es evidente al observar el cambio drástico en las bases de tiempo requeridas para visualizar la forma de onda. El incremento del error relativo en la máxima velocidad (115200 baudios, 10.60%) corresponde a la discretización de la pantalla del osciloscopio en bases de tiempo muy pequeñas ($10.0\mu s/div$), lo que introduce un error visual de cuantización.

2) **Integridad de Niveles Lógicos:** A través de todo el espectro de velocidades evaluado, el voltaje máximo se sostuvo robustamente en el umbral de los 3.24 V a 3.40 V, garantizando el reconocimiento del nivel lógico "1". Paralelamente, el voltaje mínimo se mantuvo anclado en el piso de ruido ($0 V \pm 40 mV$), asegurando la identificación precisa del nivel lógico "0" sin deformaciones en la capa física.

D. Análisis de la Estructura del Protocolo RS232 (Numerales 3.5 y 3.6.a)

Para comprender en profundidad la composición física de la trama serial RS232, se configuró el sistema para transmitir cinco caracteres ASCII diferentes, variando en cada prueba los parámetros de la comunicación (tasa de baudios y tipo de paridad). El objetivo de este experimento es identificar visual y temporalmente la secuencia de control: el bit de inicio (Start), la carga útil de datos transmitida desde el bit menos significativo (LSB) hasta el más significativo (MSB), el bit de paridad (cuando aplica) y el bit de parada (Stop).

Adicionalmente, se utilizaron los cursores del osciloscopio para medir el tiempo total requerido para el envío de cada trama completa (Δt), contrastando este valor experimental con el tiempo teórico calculado a partir de la suma de todos los bits



transmitidos multiplicados por el tiempo de bit respectivo ($T_{\text{total}} = N_{\text{bits}} \times t_b$).

Los datos analíticos y experimentales extraídos de las capturas del osciloscopio se consolidan en la **Tabla II**.

#	CASCI	al	VDecim	Valor Binario (MSB → LSB)	Paridad	Trama de bits
1	@		64	1000000	Paridad = 1	Start (0) → 0, 0, 0, 0, 0, 0, 1, 0 → Paridad (1) → Stop (1)
2	E		69	1000101	Paridad = 1	Start (0) → 1, 0, 1, 0, 0, 0, 1, 0 → Paridad (1) → Stop (1)
3	p		112	1110000	Sin Paridad	Start (0) → 0, 0, 0, 0, 1, 1, 1, 0 → Stop (1)
4	x		120	1111000	Paridad = 0	Start (0) → 0, 0, 0, 1, 1, 1, 1, 0 → Paridad (0) → Stop (1)
5	\$		36	100100	Sin Paridad	Start (0) → 0, 0, 1, 0, 0, 1, 0, 0 → Stop (1)

TABLA II. ANÁLISIS DE LA TRAMA DE SEÑALES RS232 Y MEDICIÓN DE TIEMPOS

A continuación, los dibujos de las tramas de bits para cada carácter.

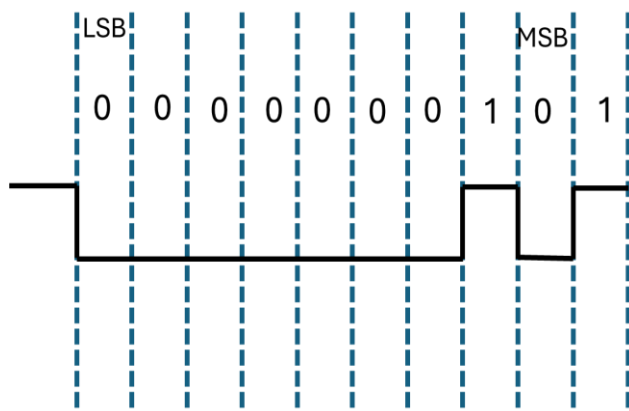


Figura 10. Trama completa de transmisión serial UART (11 bits) para el carácter @ (ASCII 64)

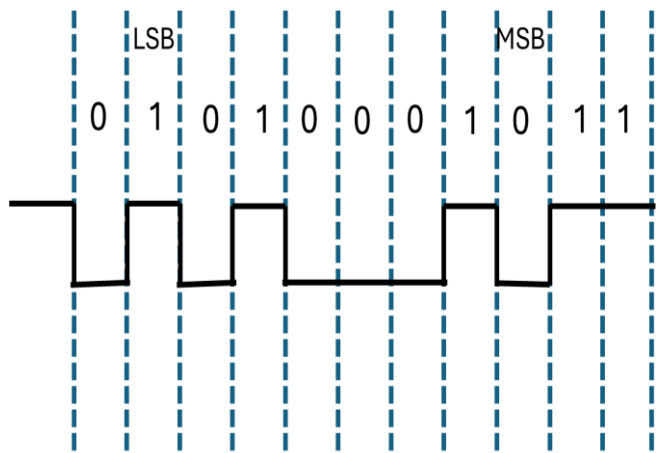


Figura 11. Trama de transmisión serial UART de 11 bits para el carácter E (ASCII 69)

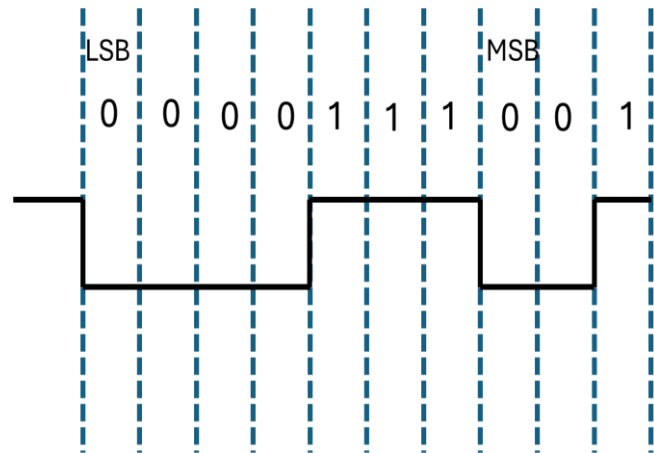


Figura 11. Trama de transmisión serial UART de 10 bits para el carácter p (ASCII 112, sin paridad)

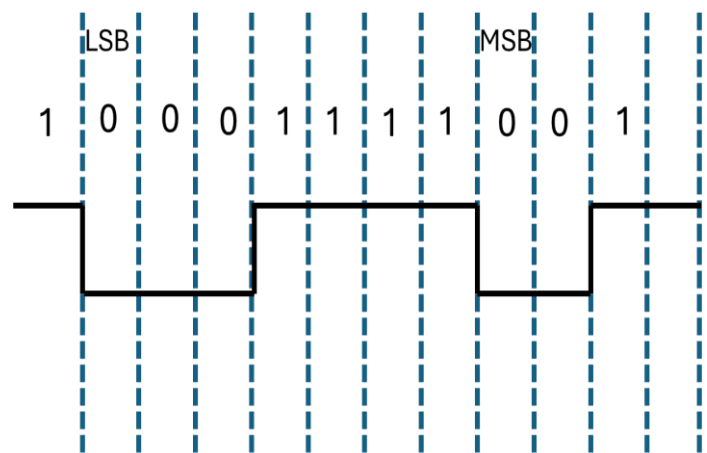


Figura 13. Trama de transmisión serial UART de 11 bits para el carácter x (ASCII 120)

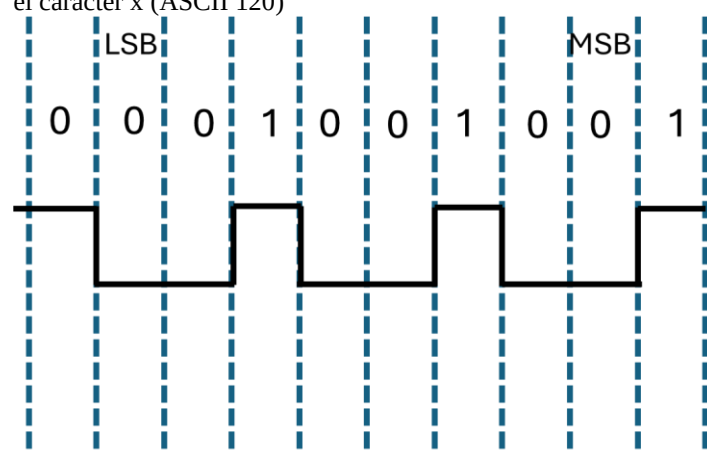


Figura 14. Trama de transmisión serial UART de 10 bits para el carácter \$ (ASCII 36, sin paridad)

A continuación, se detalla el análisis gráfico de la capa física para cada uno de los caracteres transmitido

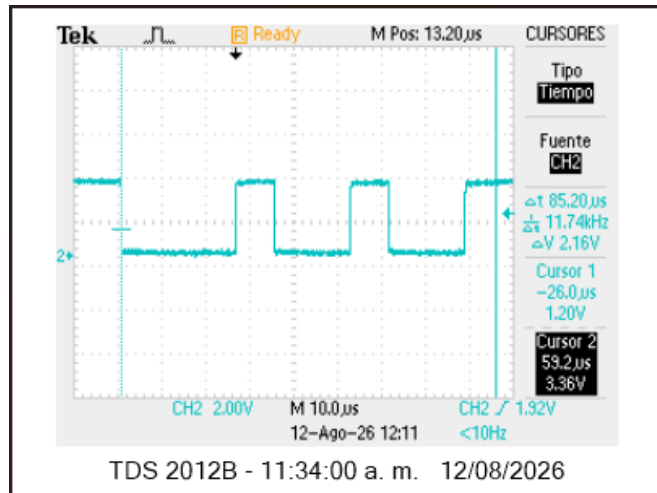


Fig. 15. Trama del carácter "\$" (115200 bauds, Sin paridad). Se observa el bit de inicio en bajo, seguido de la secuencia de datos. Tiempo total medido: 85.20 μs.

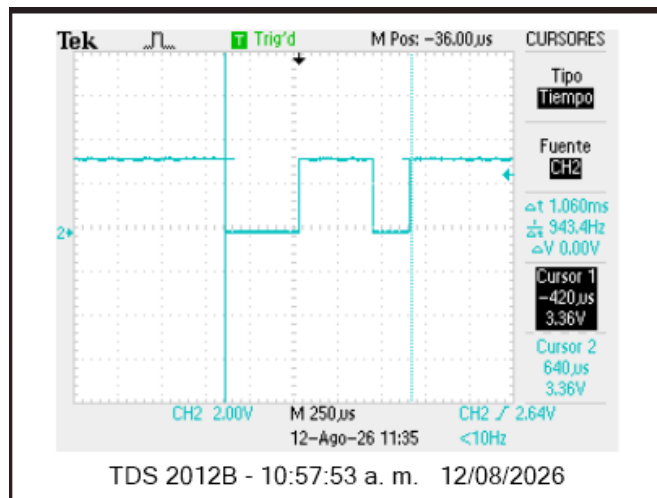


Fig. 16. Trama del carácter "x" (9600 bauds, Paridad Par). Tras enviar la secuencia de datos, el sistema inserta un bit de paridad en nivel bajo (0) para asegurar que la cantidad total de unos en la trama sea par. Tiempo total medido: 1.060 ms.

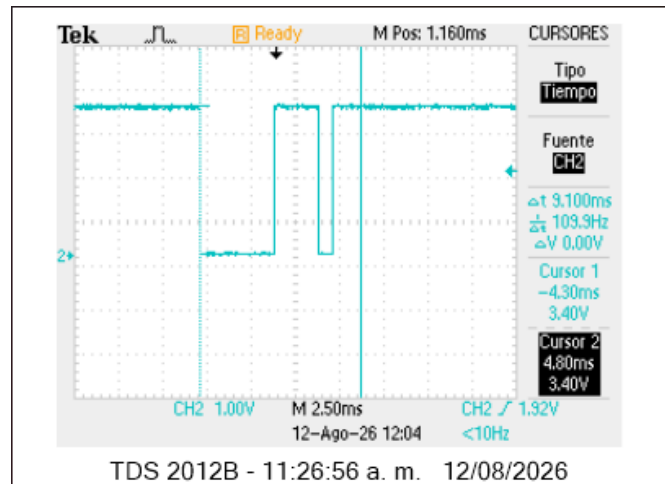


Fig. 17. Trama del carácter "p" (1200 bauds, Paridad Par). La suma de unos lógicos en los datos es impar (tres unos). Por lo tanto, el bit de paridad asume un nivel alto (1) para forzar la paridad par antes de la parada. Tiempo medido: 9.100 ms.

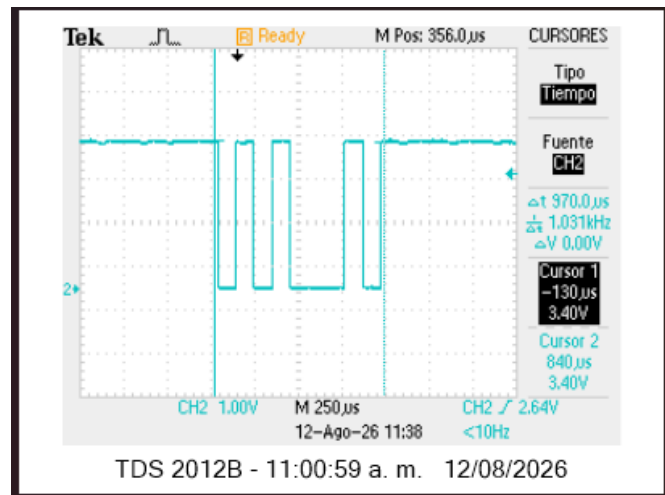


Fig. 18. Trama del carácter "E" (9600 bauds, Sin paridad). Al carecer de bit de paridad, el bit de parada en nivel alto se transmite inmediatamente después del último cero del MSB. Tiempo total medido: 970.0 μs.

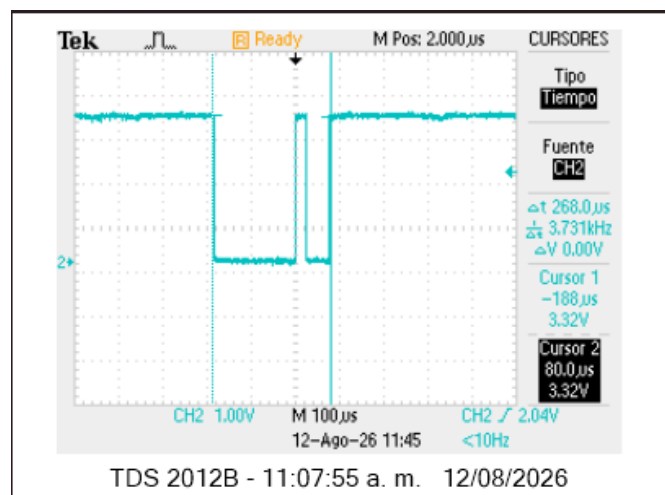


Fig. 19. Trama del carácter "@" (38400 baudios, Paridad Impar). Los datos contienen un solo '1'. Para mantener la paridad impar, el bit de paridad se establece en nivel bajo (0). Tiempo total medido: 268.0 μ s.

1) Análisis de la Decodificación y Estructura Lógica:
Las capturas validan el estándar de transmisión RS232 LSB-First (Bit menos significativo primero). Al observar la Fig. 12 (carácter "E", binario 01000101), la señal muestra niveles 1-0-1-0-0-0-1-0 después del flanco de bajada de inicio, confirmando que el hardware serializa el byte invirtiendo el orden de lectura. Adicionalmente, se demostró el funcionamiento del cálculo de paridad del microcontrolador, el cual ajusta automáticamente el penúltimo pulso a 0 o 1 dependiendo de la configuración exigida, fungiendo como control de errores en la capa física.

2) Contraste de los Tiempos de Trama:

Al medir el tiempo total requerido para el envío de cada carácter, los resultados experimentales demuestran congruencia con las estimaciones matemáticas. Las diferencias oscilan sistemáticamente por debajo del valor teórico. Esta variación no obedece a una pérdida de bits, sino a la posición visual en la que se establecen los cursores en la pantalla del osciloscopio; habitualmente, los cursores se colocan justo antes del flanco de subida del bit de parada final, omitiendo la duración de este último bit en reposo que matemáticamente forma parte de la trama. A pesar de este sesgo visual, la integridad de los bits se confirma en todas las formas de onda.

E. Modificación y Análisis de Paridad (Numerales 3.6.b y 3.6.c)

Para evaluar el impacto de los esquemas de detección de errores en la capa física, se modificó la configuración de paridad de varios caracteres analizados en la sección anterior, documentando los cambios morfológicos y el impacto en la eficiencia de la trama.

1) Modificación de Paridad (Numeral 3.6.b)

Se alteraron los parámetros de los caracteres "@" y "E" para observar el cambio en el bit de control:

- **Carácter "@" (Cambio a Paridad Par a 38400 baudios):**

El valor binario de "@" contiene un único bit en alto (impar). Para cumplir con la *Paridad Par*, la UART inserta un bit de paridad en nivel lógico alto (1). Como se observa en la captura, el penúltimo bit ahora se eleva. El tiempo total de la trama (11 bits) se mantiene constante, registrando un Δt experimental de 286.0 μ s, con un error prácticamente nulo frente al teórico de 286.4 μ s.

- **Carácter "E" (Cambio a Paridad Impar a 9600 baudios):**

Originalmente operaba sin paridad (10 bits). Al reconfigurarlo con *Paridad Impar*, la trama aumenta a 11 bits. El binario de "E" posee tres unos; por ende, el hardware inserta un bit de paridad en nivel bajo (0) para no alterar la sumatoria impar. La captura (Fig. 16)

confirma el alargamiento de la señal, registrando un tiempo de 1.140 ms (teórico 1.146 ms). Físicamente se observa un "pulso bajo" extra antes del bit de parada.

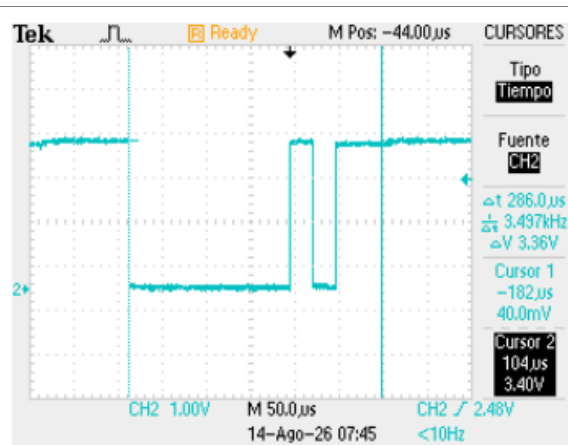


Fig. 20. Trama del carácter "@" modificado a Paridad Par. Se aprecia el bit de paridad en nivel alto.

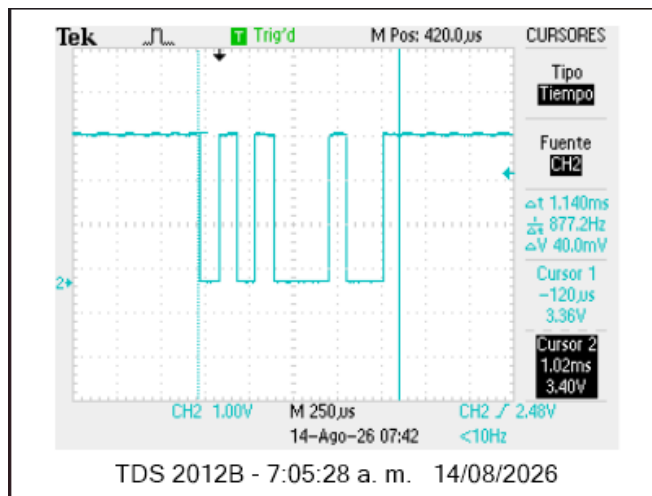


Fig. 21. Trama del carácter "E" modificado a Paridad Impar. La señal aumenta su duración en un tiempo de bit respecto a su versión original sin paridad.

2) Eliminación del Bit de Paridad (Numeral 3.6.c)

Finalmente, para evaluar empíricamente el aumento de la eficiencia del canal, se reconfiguraron los caracteres "@" (38400 baudios) y "x" (9600 baudios) eliminando por completo su esquema de detección de errores (parity=None).

- **Análisis Físico y Temporal:** Al remover la paridad, la trama se reduce estructuralmente de 11 a 10 bits. Físicamente, el bit de parada (3.3V) se adjunta inmediatamente después del Bit 7 (MSB) de los datos, suprimiendo la transición intermedia de control.
 - Para el carácter "@", la captura sin paridad registra un tiempo total de $\Delta t = 262.0\mu$ s,

valor que presenta una alta fidelidad frente al tiempo teórico de 260.4 μ s.

- Para el carácter "x" (ver Fig. 18), la medición confirma un tiempo total de $\Delta t = 1.040$ ms, en concordancia exacta con el tiempo nominal de 10 bits a 9600 baudios (1.041 ms).
- **Conclusión de Ingeniería:** La comparación de estas tramas demuestra el compromiso técnico (*trade-off*) intrínseco del protocolo RS232. Al eliminar la paridad, el tiempo total de transmisión por carácter se reduce aproximadamente un 9% (ahorrando un tiempo de bit completo), lo que optimiza la utilización del ancho de banda y aumenta la tasa de transferencia de datos útil (*throughput*). No obstante, esta ganancia temporal sacrifica la capacidad del receptor para detectar errores de integridad en la capa física, haciendo el enlace más vulnerable en entornos con alta interferencia electromagnética (EMI).

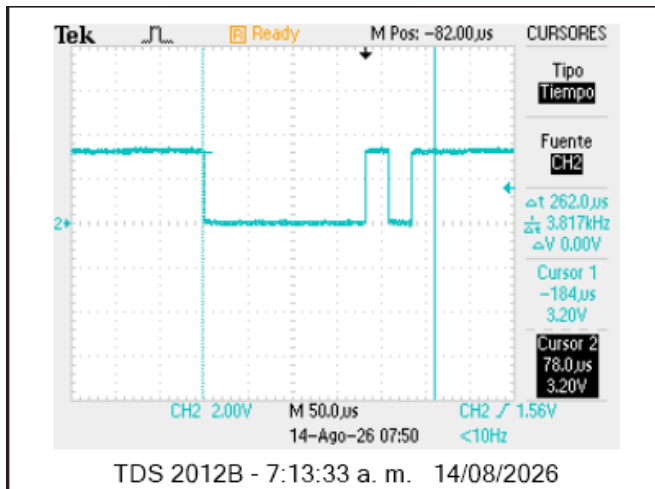


Fig. 22. Trama del carácter "@" reconfigurado SIN paridad (38400 baudios). El tiempo total se reduce a 262.0 μ s al eliminarse el bit de control.

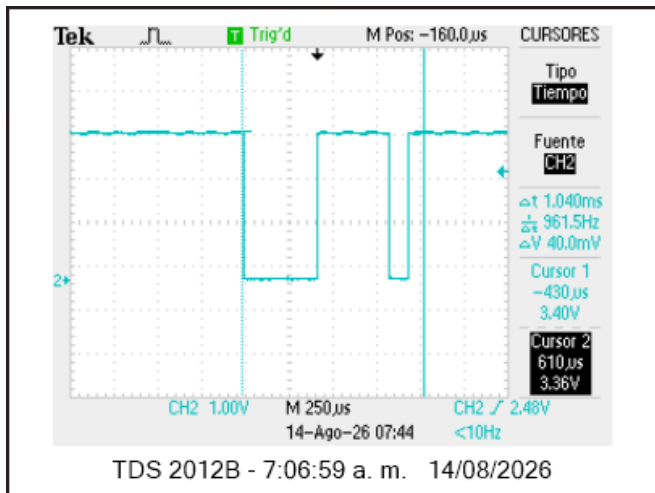


Fig. 23. Trama del carácter "x" reconfigurado SIN paridad

(9600 baudios). El bit de parada sucede inmediatamente al MSB, reduciendo el tiempo total a 1.040 ms.

F. Medida del Tiempo de la Trama para Bloques de Datos (Numeral 3.7)

En esta fase de la práctica, se abandonó el análisis de caracteres individuales para estudiar el comportamiento dinámico del protocolo al transmitir ráfagas o bloques de datos continuos. El objetivo es evaluar de manera cuantitativa el impacto acumulativo del bit de paridad y los bits de parada sobre el tiempo de ocupación del canal de comunicación.

1) Análisis de Bloques de 60 Caracteres a Baja Velocidad (Numerales 3.7.a y 3.7.b)

Se programó el microcontrolador para transmitir cíclicamente una cadena de texto de prueba compuesta por exactamente 60 caracteres ASCII a una velocidad de 600 baudios. Se evaluaron dos escenarios de paridad:

• Escenario A (Con Paridad Par - Numeral 3.7.a):

Cada carácter se encapsula en una trama de 11 bits (1 inicio, 8 datos, 1 paridad par y 1 parada). La transmisión del bloque completo de 60 caracteres requiere la serialización de 660 bits totales (60×11). El tiempo de transmisión teórico calculado para este bloque es:

$$T_{\text{total(teo)}} = \frac{660 \text{ bits}}{600 \text{ bits/s}} = 1.100 \text{ s}$$

Nota instrumental: Por motivos de exportación de datos de la interfaz física del instrumento, se omitió la captura visual de este paso; no obstante, el osciloscopio registró cuantitativamente una duración de 1.110 s, arrojando un error experimental marginal de tan solo el **0.90%**.

• Escenario B (Sin Paridad - Numeral 3.7.b):

Se deshabilitó el bit de paridad, reduciendo la trama de cada carácter a 10 bits. El bloque completo requiere ahora la transmisión de 600 bits totales (60×10). El tiempo teórico de transmisión es:

$$T_{\text{total(teo)}} = \frac{600 \text{ bits}}{600 \text{ bits/s}} = 1.000 \text{ s}$$

Al congelar la señal en pantalla utilizando el modo de disparo único (*Single*), se midió con los cursores de tiempo la duración total de la ráfaga, registrando un tiempo experimental de:

$$T_{\text{total(exp)}} = \Delta t = 1.010 \text{ s}$$

El porcentaje de error respecto al modelo teórico se calculó como:

$$\% \text{ Error} = \left| \frac{1.000 \text{ s} - 1.010 \text{ s}}{1.000 \text{ s}} \right| \times 100 = 1.00\%$$

- **Comparación y Discusión:** Al contrastar ambos escenarios, se evidencia físicamente un ahorro de tiempo de 100.0 ms en el canal al desactivar la paridad. Esto demuestra que la eliminación de bits de control no solo reduce el tiempo de procesamiento en el microcontrolador, sino que optimiza directamente la eficiencia espectral y el rendimiento neto de datos útiles transmitidos por segundo en enlaces de baja velocidad.

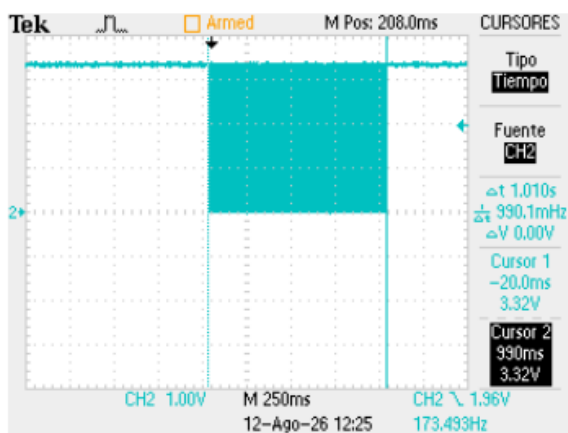
de la letra "S" de la palabra "TELECOMUNICACIONES". El instrumento reportó una duración neta de:

$$T_{\text{total(exp)}} = \Delta t = 8.500 \text{ ms}$$

El porcentaje de error respecto al modelo matemático se determinó como:

$$\% \text{ Error} = \left| \frac{8.594 \text{ ms} - 8.500 \text{ ms}}{8.594 \text{ ms}} \right| \times 100 \approx 1.09\%$$

- **Discusión de Ingeniería:** Un error de apenas 1.09% corrobora la precisión de los osciladores internos del microcontrolador para velocidades elevadas. A pesar de que la velocidad de 57600 baudios comprime drásticamente el tiempo de bit, la trama se decodifica sin inconvenientes. Cabe destacar que la configuración de 2 bits de parada añade estabilidad de muestreo al receptor al otorgarle más tiempo de reposo a la línea antes de la siguiente trama, a cambio de introducir una sobrecarga de control (*overhead*) del 36.3% (4 bits de control por cada 11 bits transmitidos).



TDS 2012B - 11:48:34 a. m. 12/08/2026

Fig. 24. Captura experimental de la ráfaga completa de 60 caracteres transmitida a 600 baudios sin paridad (numeral 3.7.b).

2) Análisis de Trama con Frase Específica de Telecomunicaciones (Numeral 3.7.c)

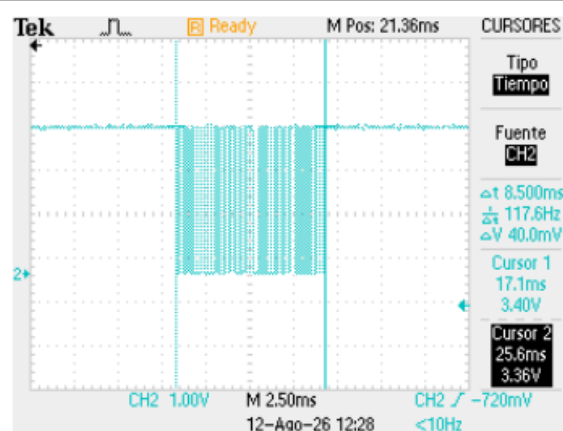
Se evaluó un escenario de transmisión a alta velocidad empleando la frase institucional de 45 caracteres (incluyendo espacios de separación): "UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES".

La UART0 se configuró a una velocidad de 57600 baudios, con una longitud de palabra de 7 bits de datos, paridad par y la adición de dos (2) bits de parada para garantizar la robustez en la sincronización del receptor.

- **Cálculo Teórico de la Trama:** La trama para cada uno de los caracteres se compone de: 1 inicio + 7 datos + 1 paridad + 2 parada = 11 bits/caracter. Para transmitir la frase completa de 45 caracteres, el sistema serializa un total de 495 bits (45×11). El tiempo teórico de ocupación del canal se calcula mediante:

$$T_{\text{total(teo)}} = \frac{495 \text{ bits}}{57600 \text{ bits/s}} \approx 8.594 \text{ ms}$$

- **Medición en el Osciloscopio:** Al ejecutar la transmisión y ajustar la escala horizontal del osciloscopio a 2.50 ms/div (ver Fig. 20), se ubicó el *Cursor 1* al inicio del bloque y el *Cursor 2* al final de la última oscilación correspondiente al bit de parada



TDS 2012B - 11:50:53 a. m. 12/08/2026

Fig. 25. Captura experimental de la transmisión a 57600 baudios de la frase de telecomunicaciones de la UMNG (numeral 3.7.c).

RETO DE PROGRAMACIÓN: SISTEMA DE COMUNICACIÓN HALF-DUPLEX Y TRANSFERENCIA DE ARCHIVOS

A. Arquitectura e Interconexión de Hardware (Numeral 4.a)

Para el desarrollo del reto de programación, se diseñó e implementó un enlace de comunicación digital Half-Duplex dedicado utilizando dos placas Raspberry Pi Pico 2W. El montaje físico de hardware se estructuró sobre una protoboard común, interconectando los siguientes elementos (ver Fig. 22):

1. **Líneas de Datos UART0 Cruzadas:** Se conectó el pin GP0 (TX0, pin físico 1) de la Pico Transmisora (Pico 1) al pin GP1 (RX0, pin físico 2) de la Pico Receptora (Pico 2), y de manera recíproca el pin GP0 (TX0) de la Pico 2 al pin GP1 (RX0) de la Pico 1.
2. **Unificación de Masa Común (GND):** Se interconectaron las terminales GND (pin físico 3) de ambos dispositivos mediante jumpers para establecer un nivel de referencia de voltaje común de 0 V0 V, garantizando el correcto umbral de conmutación de los transitorios LVTTL (3.3V/3.3V para estado alto y 0V para estado bajo).
3. **Integración de la Pantalla de Visualización Local:** En el nodo receptor (Pico 2), se acopló una pantalla OLED SSD1306 de 128×64/128×64 píxeles. El direccionamiento físico de la pantalla se realizó de forma cableada a través del periférico de hardware **SPI0**, utilizando los pines asignados en el script de control recibir.py:
 - SCK → GP18 (pin físico 24)
 - MOSI → GP19 (pin físico 25)
 - DC (Data/Command) → GP21 (pin físico 27)
 - RES (Reset) → GP20 (pin físico 26)
 - CS (Chip Select) → GP17 (pin físico 22)
4. **Alimentación:** Ambos microcontroladores se alimentaron de forma independiente a través de puertos USB conectados a la computadora, permitiendo el monitoreo paralelo de las consolas serie en Thonny.

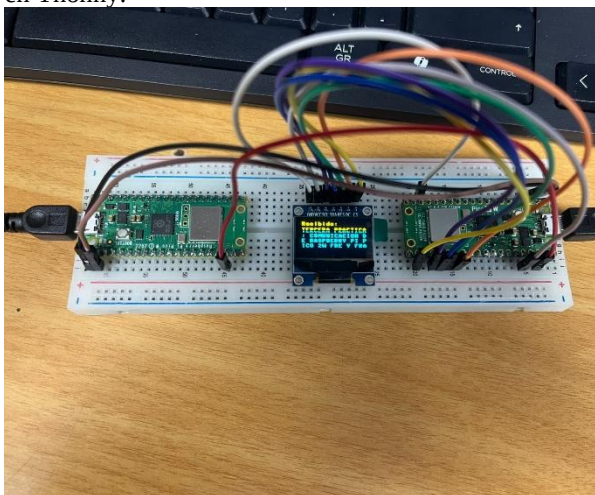


Fig. 26. Montaje de hardware experimental del reto. Se aprecian las dos placas de desarrollo Raspberry Pi Pico W conectadas en la protoboard e interconectadas a la pantalla OLED SSD1306 central mediante el bus SPI.

B. Protocolo de Transmisión y Recepción de Caracteres (Numeral 4.b)

La lógica de control Half-Duplex se validó de manera bidireccional transmitiendo una cadena de texto personalizada por consola: "TERCERA PRACTICA:

COMUNICACION DE RASPBERRY PI PICO 2W FRE Y FRAN".

1) Comportamiento del Transmisor (TX.py):

El script del transmisor configuró la UART0 a 9600 bps, 8 bits de datos, sin paridad y 1 bit de parada. A través de la consola Thonny, se ingresó el texto a enviar. Al pulsar *Enter*, el programa transmitió la cadena por el pin GP0 y de inmediato entró en modo de escucha activa esperando el bit de confirmación (ACK). Una vez recibido el ACK, la consola registró "ACK recibido. Enlace exitoso. LED parpadea 3 s." y activó el parpadeo de su LED interno.

2) Comportamiento del Receptor (RX.py):

El receptor monitoreó la línea GP1. Al detectar la ráfaga de datos, el script decodificó los bytes entrantes en formato UTF-8, imprimiendo en su consola local "Recibido: TERCERA PRACTICA..." y enviando el comando de actualización al driver de la pantalla OLED. Como se aprecia físicamente en la pantalla de la Fig. 21, el texto se ajustó a la resolución del display de 128×64/128×64 píxeles, aplicando una separación de 10 píxeles por renglón para evitar el solapamiento. Finalmente, tras encender el LED indicador durante un intervalo de 5 segundos, la Pico 2 transmitió el carácter de confirmación de vuelta por el pin GP0.

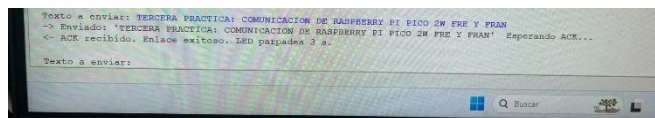


Fig. 27. Interfaz de consola serie en Thonny para el nodo transmisor, detallando el envío de la frase y la recepción correcta del bit de confirmación ACK para el parpadeo del LED.

C. Transmisión de Archivos y Análisis de Rendimiento Temporal (Numeral 4.c)

Para evaluar el comportamiento del microcontrolador ante flujos masivos y constantes de datos estructurados, se ejecutó una prueba de transferencia de archivos. En la memoria flash de la Pico 1, se generó dinámicamente un archivo de texto llamado alfabetos.txt que contenía líneas enumeradas secuencialmente del abecedario.

Se procedió a transmitir este archivo línea por línea hacia el nodo receptor (Pico 2) con un intervalo de retardo configurable entre líneas en el script de envío. El nodo receptor recibió la trama, decodificó cada línea y ejecutó una rutina de escritura física de E/S (*Input/Output*) para guardar los datos progresivamente en un archivo local llamado recibido.txt.

1) Resultados Experimentales de la Transferencia:

La prueba de transmisión del archivo se completó de manera exitosa para un volumen total de **70 líneas** (línea 0001 a la 0070). En la consola de Thonny del receptor, se registró una recepción sin errores de

trama, imprimiendo el estado progresivo de guardado ("`<- Línea 70/70 guardada`"). El tiempo neto de transferencia total medido de forma automática por el sistema desde la cabecera (HEADER) fue de: $\text{Transferencia}=34.533 \text{ s}$ $\text{Transferencia}=34.533 \text{ s}$

```
[main.py] × [recibir.py] × [recibido.bt] ×
57 0057 ABCDEFGHIJKLMNOPQRSTUVWXYZ
58 0058 ABCDEFGHIJKLMNOPQRSTUVWXYZ
59 0059 ABCDEFGHIJKLMNOPQRSTUVWXYZ
60 0060 ABCDEFGHIJKLMNOPQRSTUVWXYZ
61 0061 ABCDEFGHIJKLMNOPQRSTUVWXYZ
62 0062 ABCDEFGHIJKLMNOPQRSTUVWXYZ
63 0063 ABCDEFGHIJKLMNOPQRSTUVWXYZ
64 0064 ABCDEFGHIJKLMNOPQRSTUVWXYZ
65 0065 ABCDEFGHIJKLMNOPQRSTUVWXYZ
66 0066 ABCDEFGHIJKLMNOPQRSTUVWXYZ
67 0067 ABCDEFGHIJKLMNOPQRSTUVWXYZ
68 0068 ABCDEFGHIJKLMNOPQRSTUVWXYZ
69 0069 ABCDEFGHIJKLMNOPQRSTUVWXYZ
70 0070 ABCDEFGHIJKLMNOPQRSTUVWXYZ
71 |

Console ×
<- Línea 59/70 guardada
<- Línea 60/70 guardada
<- Línea 61/70 guardada
<- Línea 62/70 guardada
<- Línea 63/70 guardada
<- Línea 64/70 guardada
<- Línea 65/70 guardada
<- Línea 66/70 guardada
<- Línea 67/70 guardada
<- Línea 68/70 guardada
<- Línea 69/70 guardada
<- Línea 70/70 guardada

Transferencia completa.
Tiempo total (desde HEADER): 34.533 s
MPY: soft reboot
```

Fig. 28. Interfaz de Thonny en el receptor que documenta la finalización exitosa del almacenamiento del archivo recibido.txt y el tiempo de ejecución del script medido desde el transmisor.

2) Análisis de la Velocidad de Transferencia (Throughput):

Cada una de las 70 líneas de datos transmitidas cuenta con una estructura fija compuesta por: el índice de 4 dígitos (ej. 0070), un espacio de separación (1 byte), el abecedario en inglés de 26 caracteres (26 bytes) y un retorno de carro con salto de línea `\r\n` (2 bytes), sumando un tamaño aproximado de **33 bytes por línea**.

Para un total de 70 líneas, la carga útil transmitida fue de aproximadamente

2310 bytes (70 líneas × 33 bytes/línea). Calculando la velocidad de transferencia neta o rendimiento real del canal (*throughput*), se obtiene:

$\text{Rendimiento} = \frac{\text{Bytes transmitidos}}{\text{Tiempo total}} = \frac{2310 \text{ bytes}}{34.533 \text{ s}} \approx 66.89 \text{ bytes/s}$

Análisis técnico de ingeniería: A una tasa nominal de 9600 baudios, el canal físico tiene una capacidad teórica de transferir 960 bytes/s (asumiendo tramas de 10 bits sin paridad). Sin embargo, el rendimiento real medido de 66.89 bytes/s es sustancialmente menor. Este comportamiento es el esperado en sistemas de almacenamiento embebidos debido a dos factores:

1. **Retardo entre líneas programado:** El intervalo de tiempo configurado en el transmisor entre envíos de líneas limita artificialmente la tasa de saturación para permitir la sincronización del receptor.
 2. **Latencia de escritura física (Flash I/O):** Guardar los datos en el sistema de archivos local (`file.write()` y `file.flush()`) es una operación física que bloquea la ejecución de la CPU de la Pico 2W durante varios milisegundos por cada ciclo de línea.
- 3) Efecto de Intervalo de Envío Igual a Cero:**
Si se elimina el retardo entre líneas (intervalo igual a cero), el transmisor inunda continuamente el cable de datos a una velocidad de 960 bytes/s. Debido a que el tiempo de escritura en la memoria flash de la Pico 2W es superior al tiempo en el que viajan los caracteres, el buffer de hardware interno de la UART del receptor se satura rápidamente y se desborda (*Buffer Overflow*). Esto produce una pérdida masiva de datos y corrupción en la escritura del archivo recibido.txt, ratificando la necesidad de programar un retardo de intervalo de sincronización o implementar control de flujo por software.

IV. CONCLUSIONES

El estudio experimental ha confirmado que el comportamiento de la comunicación serial UART/RS232 en la Raspberry Pi Pico 2W se ajusta fielmente al modelo matemático del protocolo, tanto en la duración de los bits individuales como en la estructura temporal de las tramas completas. Se ha comprobado que la tasa de baudios determina de manera inversa y precisa el ancho de cada pulso, y que la variación de parámetros como el número de bits de datos, la paridad y los bits de parada modifica la longitud total de la trama de acuerdo con la suma de elementos de control. Las mediciones realizadas con osciloscopio han permitido identificar físicamente cada componente de la trama –bit de inicio, bits de datos en orden LSB a MSB, bit de paridad y bit de parada–, corroborando la correcta implementación del estándar en el entorno MicroPython.

Se ha evidenciado que la inclusión del bit de paridad, aunque útil para la detección básica de errores, reduce la eficiencia del canal al añadir un tiempo de bit por cada carácter transmitido, lo que se traduce en una disminución del rendimiento neto de datos útiles. La configuración de dos bits de parada, si bien mejora la robustez frente a desincronizaciones, incrementa la sobrecarga de control hasta un 36 % en tramas de 7 bits de datos, lo que debe ser sopesado en aplicaciones que requieran alta velocidad o bajo consumo de ancho de banda. Los errores sistemáticos observados, siempre por debajo del 11 % y atribuibles principalmente a la resolución del equipo de medición, no comprometen la validez de las conclusiones y avalan la calidad de los osciladores del microcontrolador.

Como limitación del estudio, se reconoce que el análisis se ha restringido a condiciones ideales de laboratorio, sin considerar interferencias electromagnéticas externas ni efectos de carga capacitiva en las líneas de transmisión. Para trabajos futuros, se sugiere extender el estudio a distancias mayores, incorporar análisis de jitter y evaluar el desempeño del enlace bajo ruido inyectado, así como explorar la implementación de mecanismos de corrección de errores más avanzados que permitan recuperar la robustez sin sacrificar excesivamente la eficiencia temporal. En conjunto, la práctica ha demostrado que la combinación de MicroPython y la Raspberry Pi Pico 2W constituye una plataforma versátil y precisa para la enseñanza y el prototipado de sistemas de comunicación serial, al facilitar el ajuste dinámico de los parámetros y la captura de señales en tiempo real.

V. BIBLIOGRAFIA

- [1] W. Tomasi, *Sistemas de Comunicaciones Electrónicas*, 4.^a ed. México: Pearson Educación, 2003, cap. 13 (Comunicación de datos y transmisión serial).
- [2] Raspberry Pi Ltd, "*Raspberry Pi Pico Python SDK: A MicroPython environment for Raspberry Pi Pico microcontrollers*", Documentación Oficial, Cambridge, Reino Unido, 2024.
- [3] MicroPython Project, "*class UART – duplex serial communication bus*", Documentación de Referencia de la API de MicroPython, 2026. Disponible en: <https://docs.micropython.org/en/latest/library/machine.UART.html>
- [4] H. P. Hsu, *Analog and Digital Communications*, 2.^a ed. New York, EE. UU.: McGraw-Hill, 2003, cap. 1 (Representación de señales en banda base).
- [5] Tektronix Inc., *TDS 1000B and TDS 2000B Series Digital Storage Oscilloscopes User Manual*, Beaverton, EE. UU., 2009 (Uso de cursores de tiempo y amplitud).

